

ArcRelay Execution Roadmap

A Day-by-Day Masterplan: From Testnet MVP to Live Commercial Pilot on Arc Mainnet

Target Network: Arc Mainnet (Circle Ecosystem / Native USDC Gas)

Execution Window: 60 Days (September 24 – November 24, 2026)

Lead Founder: @kryptbamba | **Engineering Stack:** Antigravity + Async Python + EVM

Capital Requirement: Under \$100 Initial Out-of-Pocket

The Core Directive:

*Arc Mainnet launched on September 16, 2026. The network delivers USDC as native gas, eliminating the volatile token friction that causes 82% of non-crypto users to abandon Web3 apps. This eBook is the definitive execution roadmap for building **ArcRelay**—the turn-key 'EZ-Pass' infrastructure that allows apps to sponsor user transactions automatically from corporate USD accounts. Follow this guide to move systematically from code to ecosystem grant to paying clients.*

Chapter 1: The Arc Genesis Window

On traditional chains, Account Abstraction (ERC-4337) is plagued by three fatal flaws: corporate treasuries cannot legally hold volatile gas tokens (ETH/POL), paymasters suffer from price oracle slippage, and collected stablecoins must be continually swapped on decentralized exchanges with high fee friction.

The Arc Structural Edge:

- **Native USDC Gas:** 1 USDC of network fee = 1 USDC deducted from the corporate deposit. Zero exchange rate risk, zero DEX swaps, zero slippage.
- **Sub-Second Finality:** Deterministic blocks eliminate mempool front-running, sandwich attacks, and uncertain confirmation times.
- **190+ Launch Partners:** Neobanks, payroll providers, and gaming studios are actively building on Arc right now. All of them need invisible gas rails.
- **The 90-Day Window:** Big generalist providers (Alchemy, Pimlico) are distracted across 50+ chains. We have a golden 90-day window to become the default vertical standard on Arc.

Chapter 2: The 60-Day Sprint Breakdown

Sprint & Phase	Key Deliverables	Target Milestone
Sprint 1 Days 1–14 <i>(The Engine)</i>	• Compile ArcRelayPaymaster.sol • Connect to Arc Testnet faucet • Build async Python policy router • EIP-712 off-chain signature tests	First Gasless Tx Executed and verified live on Arc Testnet.
Sprint 2 Days 15–30 <i>(The Product)</i>	• Secure domain (arcrelay.io) • Launch B2B Landing Page • Publish 3-line Python/TS SDK • Wire /sponsor-demo to Telegram	Public Developer Demo Live website and interactive Telegram demo bot.
Sprint 3 Days 31–45 <i>(The Grants)</i>	• Formal Arc Ecosystem Grant app • Circle Ventures outreach • Pitch 15 Arc launch partners • Offer free sponsored gas credits	Grant Submission & Pilot 5 ecosystem apps signed for closed beta.
Sprint 4 Days 46–60 <i>(Commercial)</i>	• Deploy to \$15/mo cloud server • Arc Mainnet contract deployment • Production monitoring alerts • Activate 5%–8% volume billing	First Revenue Live mainnet transactions processing daily.

Chapter 3: Technical Blueprint & Architecture

ArcRelay is built on three tightly integrated layers: on-chain smart contracts, an off-chain async policy engine, and client SDKs:

Layer 1: On-Chain Smart Contract (`ArcRelayPaymaster.sol`)	Written in Solidity conforming to ERC-4337 v0.7. Holds client USDC tanks directly at the Arc EntryPoint. Validates user transactions via EIP-712 off-chain cryptographic signatures signed by our backend. Debits USDC with zero token conversion.
Layer 2: Off-Chain Policy Daemon (`policy_engine.py`)	High-throughput Python async service using `httpx` and `asyncio`. Receives UserOps via JSON-RPC, checks developer spending limits (e.g. max \$5/user/month), verifies client account solvency, signs the approval hash, and submits the batch to Arc validator nodes.
Layer 3: Client Developer SDK (`@arcrelay/sdk`)	A dead-simple 3-line library for mobile and web apps. DApp developers wrap standard Web3 transactions with `arcrelay.sponsor(tx)` and their users never see a gas fee prompt.

Chapter 4: The Founder's 'Build in Public' Playbook

The Strategy: Secure the brand handle (@ArcRelayHQ) immediately, but do the primary storytelling through @kryptbamba. People and ecosystem funds back authentic human founders sharing technical progress, not faceless corporate logos.

Timing	Target Post Content & Objective
Day 1–2 <i>The Teaser</i>	Post 1 (The Hook): 'Arc Mainnet launched with native USDC gas, and most people don't realize how massive this is for mainstream adoption. Over the next few weeks, I'm building something to make transactions on Arc 100% invisible and gasless. Stay tuned. ■ #Arc #Circle #BuildInPublic'
Day 7–10 <i>The Reveal</i>	Post 2 (The Solution): 'Introducing @ArcRelayHQ: The zero-friction gasless infrastructure built natively for Arc. Think of it like an EZ-Pass for apps. Companies pre-fund their gas tank in clean USD, and their users send money or trade with ZERO fee friction. Architecture dropping soon.'
Day 14–16 <i>The Proof</i>	Post 3 (Working Testnet Demo): Share a 15-second screen recording showing our first real gasless transaction on Arc Testnet. Tag @arc, @circle, and key ecosystem contributors.
Day 25–30 <i>The SDK</i>	Post 4 (Developer Outreach): 'Calling all devs building on Arc! We just open-sourced the 3-line @ArcRelayHQ SDK. Sponsoring your users' transactions now takes less than 2 minutes to integrate. Looking for 5 pilot apps to give free gas credits to. DM me!'

Chapter 5: Risk Management & Unit Economics

- 1. Zero Capital Risk for Us:** Client applications pre-fund their own corporate gas tanks. ArcRelay never lends capital or extends credit. If a client app's deposit reaches \$0, sponsorship pauses automatically until replenished.
- 2. Anti-Spam Rate Limiting:** Our off-chain policy engine enforces strict per-user quotas (e.g., maximum \$1.00 sponsored gas per wallet per day). Rogue users cannot drain a client's tank.
- 3. Multi-Chain Portability:** Because our codebase strictly adheres to the EVM standard, our contracts and Python engine can deploy to Base, Arbitrum, or Polygon in under 24 hours if ecosystem conditions ever require pivot.

Operational Phase	Monthly Gas Volume	Markup Rev (6%)	SaaS Subscription	Net Monthly Run-Rate
Pilot (Month 3)	\$25,000	\$1,500	\$1,000	\$2,500 / mo
Growth (Month 6)	\$150,000	\$9,000	\$6,500	\$15,500 / mo
Scale (Month 12)	\$600,000	\$36,000	\$18,000	\$54,000 / mo

Chapter 6: The Founder's Action Checklist

- [] **Today (Day 1):** Initialize the `arcrelay-core` project directory on your PC. Reserve the @ArcRelayHQ handle on X/Twitter.
- [] **Days 2–4:** Draft and compile `ArcRelayPaymaster.sol`. Write local unit tests for USDC fee deduction.
- [] **Days 5–7:** Connect Python policy engine to Arc Testnet. Run the first end-to-end gasless transaction test.
- [] **Days 8–10:** Publish Post 1 on @kryptbamba sharing the vision of invisible gas on Arc.
- [] **Days 11–14:** Record a 15-second demo video of the working testnet transaction. Prepare Grant Pitch deck.

***Final Founder Note:** You have the strategy, the technical blueprint, the initial code stack, and the first-mover timing. You don't need millions in funding to begin. Focus strictly on Sprint 1: get the first transaction working on testnet. Everything else flows from proof.*